

What we have covered till now ?

LED
ADC
UART
Timers

EV Control Module

ev_control.c / ev_control.h

- Physics-based inertia model & speed computation
- SOC estimation via energy integration
- Range estimation with drive-mode efficiency
- ECO / NORMAL / SPORT torque scaling
- Regenerative braking & motor power (kW)



Module Overview — ev_control.h

Public API, data structures, and constants

State Data

- speed_kmh
- motor_torque (Nm)
- soc (0–100 %)
- range_km
- power_kw
- regen_level
- drive_mode
- motor_temp

Key Constants

- MAX_SPEED = 200 km/h
- MAX_TORQUE = 200 Nm
- REGEN_MAX = -80 Nm
- BRAKE_THRESH = 5%
- BATTERY_CAP_kWh
- DRAG_COEFF = 5.0
- MASS_FACTOR = 450
- SIM_SCALE

Functions

- ev_control_init()
- ev_update_speed()
- ev_update_soc()
- ev_estimate_range()
- ev_set_drive_mode()
- ev_apply_regen()

Speed — Lumped Inertia Model (Euler Integration)

Linearised drag in Nm | $\Delta v = \text{accel} \times dt \times 3.6$ | Clamped to [0, 200] km/h

```
accel = ( motor_torque - drag_Nm ) / EV_MASS_FACTOR  
        accel * dt * 3.6  
→ speed_kmh +=
```

motor_torque

Commanded torque (Nm)
From throttle $\times$ MAX_TORQUE
 $\times$ mode_scale
Negative during regen

drag_Nm

Lumped drag (Nm)
 $\text{drag} = \text{speed_ms} \times$
 DRAG_COEFF
Linear approx of aero
+ rolling resistance
 $\text{DRAG_COEFF} \approx 5.0 \text{ N}\cdot\text{s/m}$

MASS_FACTOR

Effective inertia (kg·m)
 $= \text{vehicle_mass} \times \text{wheel_r}$
e.g. $1500 \text{ kg} \times 0.3 \text{ m}$
 $= 450 \text{ kg}\cdot\text{m}$
Gives accel in m/s^2

CLAMP

Speed bounds check
 $\text{CLAMP}(v, 0, 200 \text{ km/h})$
Prevents negative speed
and enforces top speed
limit in simulation

```
float speed_ms = ev->speed_kmh / 3.6f; float drag_Nm = speed_ms * EV_DRAG_COEFF; ev->speed_kmh = CLAMP(ev->speed_kmh + accel  
* dt * 3.6f, 0.0f, EV_MAX_SPEED_KMH);
```

SOC Estimation — Energy Integration

P_mech + P_copper_loss | Scaled by EV_SIM_SCALE for demo speed

$$P_{kW} = (\tau \times v_{ms} / 1000) + (\tau / \tau_{max})^2 \times 5.0$$
$$\Delta SOC = (P_{kW} \times dt) / (CAP_{kWh} \times 3600) \times 100 \times SIM_SCALE$$

Computation Steps

- 1. $v_{ms} = speed_kmh / 3.6$
- 2. $P_{mech} = motor_torque \times v_{ms} / 1000$ (kW)
- 3. $torque_ratio = torque / EV_MAX_TORQUE_NM$
- 4. $P_{loss} = torque_ratio^2 \times 5.0$ (copper loss kW)
- 5. $power_kw = P_{mech} + P_{loss}$
- 6. $delta_soc = power_kw \times dt / (CAP \times 3600) \times 100 \times SCALE$
- 7. $soc = CLAMP(soc - delta_soc, 0, 100)$

Why Copper Loss Term?

At $v = 0$: $P_{mech} = 0 \rightarrow$ SOC never drains at standstill.
Copper loss ($I^2R \propto torque^2$) adds up to 5 kW even when stationary — models real motor current draw.

EV_SIM_SCALE — Demo Tuning

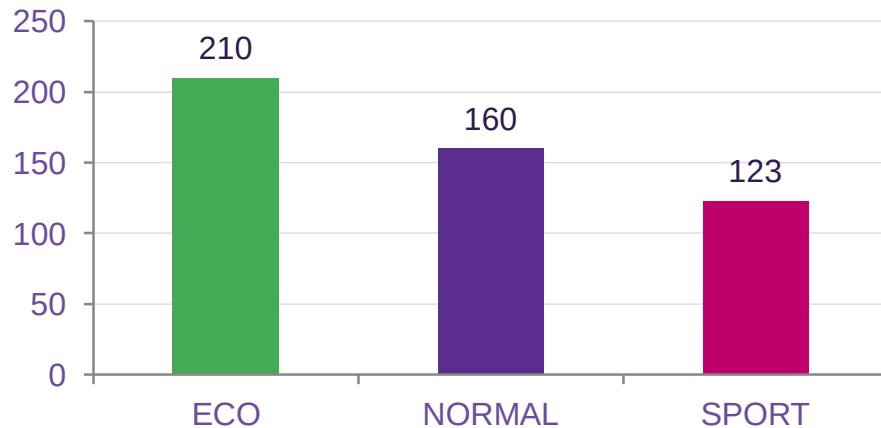
SIM_SCALE	Full drain time
1.0	~2.7 hours (real)
32.0	~5 minutes
160.0	~1 minute

Range Estimation

Remaining distance from SOC, battery capacity, and drive-mode efficiency

$$\text{Range (km)} = (\text{SOC\%} / 100) \times E_{\text{battery_kWh}} / \eta_{\text{mode}} \times \text{km_per_kWh}$$

Estimated Range by Drive Mode



Efficiency Coefficients

Mode	η Factor	km/kWh (est.)
ECO	$\div 0.6$ (most range)	~ 8.0
NORMAL	$\div 1.0$ (baseline)	~ 5.5
SPORT	$\div 1.3$ (least range)	~ 4.2

Drive Modes — Torque Scaling

ECO · NORMAL · SPORT: three tiers of torque multiplier

ECO × 0.6

Restricts torque to 60% of request.
Prioritizes range and efficiency.
Smooth, gentle acceleration.
Suitable for city/highway cruise.

Peak Torque

120 Nm (at full throttle)

Daily commute, max range

NORMAL × 1.0

Full torque passthrough.
Balanced performance & range.
Default operating mode.
Standard driver experience.

Peak Torque

200 Nm (at full throttle)

General driving

SPORT × 1.3

Boosts torque by 30% (capped at max).
Sharp, responsive acceleration.
Higher power draw → faster SOC drop.
Thermal monitoring recommended.

Peak Torque

200 Nm (clamped to max)

Performance, overtaking

Regenerative Braking

Negative torque PWM when brake pedal > 5% | Max -80 Nm

Control Logic

Read brake pedal ADC (0–100%)

If brake% > EV_REGEN_THRESHOLD_PCT (5%)

$\text{regen_level} = \text{brake_pedal} \times 0.7$ (0–70% scale)

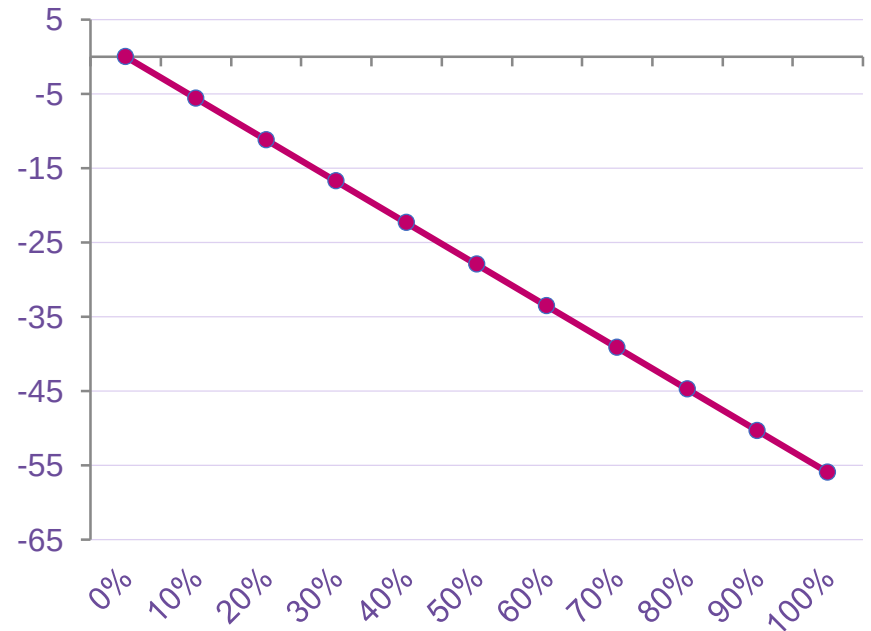
$\text{motor_torque} = -(\text{regen_level} / 100) \times \text{REGEN_MAX_NM}$

At 100% brake $\rightarrow$ torque = $-0.7 \times 80 = -56$ Nm

Negative torque $\rightarrow$ negative P_{mech} $\rightarrow$ SOC increases

$\text{brake\%} \leq 5\% \rightarrow$ regen inactive, normal torque resumes

Regen Torque vs Brake Pedal % (70% scale)



Speed Integration & Motor Power Computation

Euler step in km/h | P_mech + Copper Loss | Hard clamp at 200 km/h

Speed — Euler Integration

```
speed_ms = speed_kmh / 3.6f

drag_Nm = speed_ms * EV_DRAG_COEFF

accel = (torque - drag_Nm)
        / EV_MASS_FACTOR

speed_kmh = CLAMP(
    speed_kmh + accel*dt*3.6f,
    0.0f, EV_MAX_SPEED_KMH);
```

Motor Power = Mech + Copper Loss

```
v_ms = speed_kmh / 3.6f

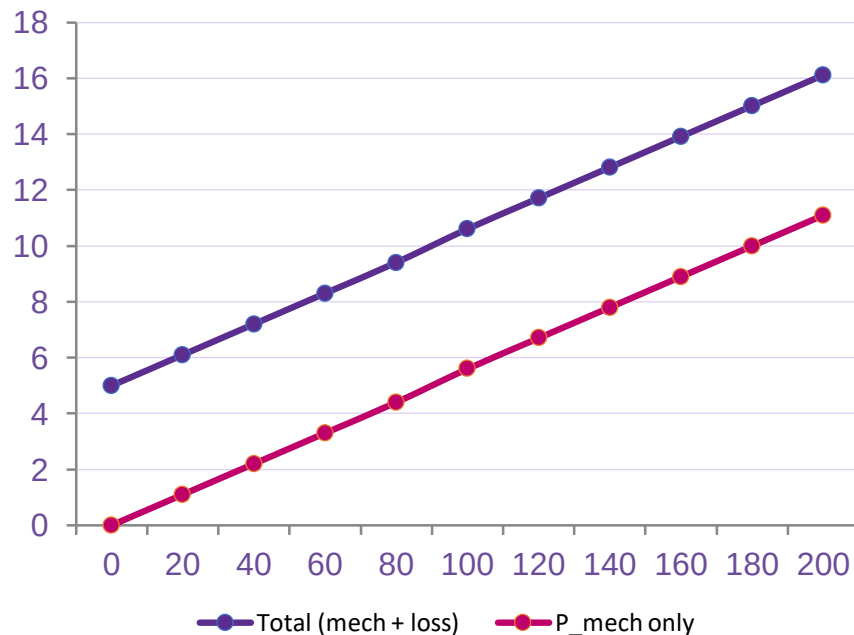
p_mech_kw = torque * v_ms / 1000.0f

torque_ratio = torque / EV_MAX_TORQUE_NM

p_loss_kw = ratio2 * 5.0f

power_kw = p_mech_kw + p_loss_kw;
```

Motor Power vs Speed (NORMAL, $\tau=200$ Nm)



Code Walkthrough — ev_control.h

Typedef, enums, and function prototypes

```
typedef enum {
    DRIVE_ECO      = 0,    /* × 0.6 torque */
    DRIVE_NORMAL   = 1,    /* × 1.0 torque */
    DRIVE_SPORT    = 2,    /* × 1.3 torque */
} DriveMode_t;

typedef struct {
    float    speed_kmh;      /* current vehicle speed */
    float    torque_Nm;     /* commanded motor torque */
    float    soc_pct;       /* state of charge 0-100 */
    float    range_km;      /* estimated range */
    float    power_kW;      /* instantaneous power */
    DriveMode_t mode;
    uint8_t  regen_active;
} EV_State_t;

void  ev_control_init    (EV_State_t *ev);
void  ev_update_speed    (EV_State_t *ev, float throttle, float dt_s);
void  ev_update_soc      (EV_State_t *ev, float dt_s);
float ev_estimate_range  (const EV_State_t *ev);
void  ev_set_drive_mode  (EV_State_t *ev, DriveMode_t mode);
void  ev_apply_regen     (EV_State_t *ev, float brake_pct);
```

Code Walkthrough — ev_control.c

Torque · Regen · Speed · Power · SOC

Torque & Regen

```
/* Step 1 - throttle */
torque = (accel/100.0f) * MAX_NM * mode_scale;

/* Step 2 - regen */
if (brake > REGEN_THRESHOLD_PCT) {
    regen_level = brake * 0.7f;
    motor_torque = -(regen_level/100.0f)
        * REGEN_TORQUE_MAX_NM;
}
```

Speed — Euler Step

```
/* Step 3 - inertia model */
float speed_ms = speed_kmh / 3.6f;
float drag_Nm = speed_ms * EV_DRAG_COEFF;
float accel = (motor_torque - drag_Nm)
    / EV_MASS_FACTOR;

speed_kmh = CLAMP(
    speed_kmh + accel * dt * 3.6f,
    0.0f, EV_MAX_SPEED_KMH);
```

Power (Step 4) & SOC Integration (Step 5)

```
float v_ms = speed_kmh / 3.6f;
float p_mech_kw = motor_torque * v_ms / 1000.0f; /* mechanical power */
float torque_ratio = motor_torque / EV_MAX_TORQUE_NM;
float p_loss_kw = torque_ratio * torque_ratio * 5.0f; /* I²R copper loss, ≤5 kW */
ev->power_kw = p_mech_kw + p_loss_kw;

float delta_soc = (ev->power_kw * dt) / (EV_BATTERY_CAPACITY_KWH * 3600.0f) * 100.0f * EV_SIM_SCALE;
ev->soc = CLAMP(ev->soc - delta_soc, 0.0f, 100.0f);
```

Motor Thermal Model — Warm-up / Cool-down

First-order Newton cooling | 5% of P_kW → heat | Clamp [25°C, 130°C]

$$dT/dt = |P_kW| \times 0.05 - (T_motor - 25) \times 0.01 \rightarrow T_new = CLAMP(T + dT \times dt, 25, 130)$$

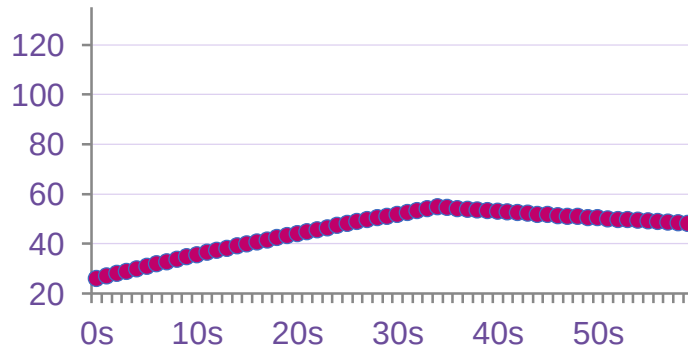
Heat Input

$|P_kW| \times 0.05$
5% of motor power
converted to heat.
Applies during both
powering and regen.

Cooling Term

$(T - 25) \times 0.01$
Newton's law of
cooling. Rate $\propto$ temp
above ambient 25°C.
Coeff 0.01 → gradual.

Temp: 20 kW load then idle



```
float thermal_power = fabsf(ev->power_kw) * 0.05f; /* 5% loss as heat */
float cooling        = (ev->motor_temp - 25.0f) * 0.01f; /* Newton cooling */
ev->motor_temp      = CLAMP(ev->motor_temp + (thermal_power - cooling)*dt, 25.0f, 130.0f);
```

Module Summary

ev_control.c / ev_control.h — Key Takeaways

- Torque:** $\text{throttle\%} \times \text{MAX_NM} \times \text{mode_scale}$ (ECO×0.6 / NRM×1.0 / SPT×1.3)
- Regen:** $\text{brake} > 5\% \rightarrow \text{torque} = -(\text{brake} \times 0.7 / 100) \times \text{REGEN_MAX_NM}$
- Speed:** $\text{drag_Nm} = \text{speed_ms} \times \text{DRAG_COEFF}$; $\Delta v = (\tau - \text{drag}) / \text{MASS_FACTOR} \times dt \times 3.6$
- Power:** $P_{\text{kW}} = \tau \times v_{\text{ms}} / 1000 + (\tau / \tau_{\text{max}})^2 \times 5$ (mech + copper loss)
- SOC:** $\Delta \text{SOC} = P_{\text{kW}} \times dt / (\text{CAP_kWh} \times 3600) \times 100 \times \text{EV_SIM_SCALE}$
- Range:** $\text{range_km} = \text{remaining_Wh} / \text{eff_Wh_per_km}$ (ECO vs OTHER mode)
- Thermal:** $dT = (|P| \times 0.05 - (T - 25) \times 0.01) \times dt \rightarrow \text{CLAMP } [25^\circ\text{C}, 130^\circ\text{C}]$